```
  And I type <password> in the password field
  Then I am logged in
Examples:
  | login| email             | password| password_confirmation|
  | alpha| alpha@example.com| alpha1  | alpha1               |
  | beta | beta@example.com | beta12  | beta12               |
```

The steps inside the outline are regular Cucumber steps. When the outline runs, it runs once for each data row in the Examples table, with the <login> syntax indicating that a value from that table should be inserted in the row.

All these features give you a tremendous amount of power. I advise you to use it sparingly. It's tempting to use these tools to reduce duplication or make your steps more general. But the flip side is that you're often declaring implementation data explicitly in the Cucumber file rather than implicitly in the step definition.

There are at least three problems with these more advanced Cucumber steps:

- All the flexibility can make for complicated step definitions. Since Cucumber depends on the step definitions doing exactly what they say they're going to do 100% of the time, complex step definitions are bad because they're more likely to contain errors. Debugging step definitions will make you question your life choices. Keeping step definitions simple makes Cucumber easier to manage.
- Putting a lot of codelike things—including data, attribute names, and CSS selectors—in Cucumber feature files makes them hard to read and parse. Since the point of Cucumber is to be natural language–like, writing unreadable steps defeats the purpose.
- Similarly, but more subtly, putting data in the feature file robs the feature file of its ability to declare intent. What is the point of the line Given a user that has been on the site for 2 months? It's hard to tell. Given a user that has been on the site long enough to be trusted is much more clear and explains why the step exists. This is a case where specifics imply greater meaning to your somewhat arbitrary data choices than they deserve.

## Is Cucumber Worth It?

That depends on what "it" is. Cucumber is a very helpful acceptance-test framework, provided your expectations of it are reasonable. It's a lousy unit-test framework, and if you try to use it for unit testing you'll hate it and possibly stop eating salads to avoid cucumbers, which is bad for your health.

I use Cucumber for the relatively minimal goals of being able to write my integration tests at the level of user behavior and being able to easily separate